

Elementare Datentypen und Referenzen – Teil 1

Primitive Typen und Werte

Programmierung 1

PIB-PR1 / DFIW-PRG1

Software Engineering und Software Quality Assurance { SESQA }

Prof. Dr. Martin Burger { 10. November 2025 }

noan



Plan der Präsenzveranstaltungen

VW	KW	 Inhalte in der „Vorlesung“	Mo	Di	Mi	Do
1	42	Willkommen und erste Orientierung, Lebenslanges Lernen				
2	43	Einstieg in Java (Kap. 1)				
3	44	Teamarbeit				
4	45	Klassen und Objekte (Kap. 2)				
5	46	Elementare Datentypen und Referenzen (Kap. 3)				
6	47	Methoden benutzen Instanzvariablen (Kap. 4)				
7	48	Ein Programm schreiben (Kap. 5)				
8	49	Die Java-API kennenlernen (Kap. 6)				
9	50	Vererbung und Polymorphie (Kap. 7)				
10	51	Interfaces und abstrakte Klassen (Kap. 8)				
	52	Vorlesungsfreie Zeit (Jahreswechsel)				
	01	Vorlesungsfreie Zeit (Jahreswechsel)				
11	02	Konstruktoren und Garbage Collection (Kap. 9)				
12	03	Zahlen und Statisches (Kap. 10)				
13	04	Exception-Handling (Kap. 13)				
14	05	Serialisierung und Datei-I/O (Kap. 16)				
15	06	Fragen und Antworten (Puffer)				

„Kein Plan 
überlebt den
ersten
Kontakt  mit
der Realität .

Frei nach Helmuth Karl
Bernhard von Moltke

Elementare Datentypen und Referenzen

 Plan für diese Woche

 Typsicherheit

 Variablendeklaration

 Elementartypen

 Literale Werte

 Schlüsselwörter

 Pair Programming

 Objektreferenzen

 Objekte auf dem Heap

 Arrays

 Übungen

 Pair Programming

 Spaß

📖 Elementare Datentypen und Referenzen

🔗 Verbindungen herstellen

👤 Tauschen Sie sich mit einer Nachbar:in aus!

🦒 Wann geben Sie den Typ Ihrer Variablen an?

📧 Was sind die Regeln für die Deklaration einer Variablen?

100 Was ist der Unterschied zwischen `boolean`, `byte` und `int`?

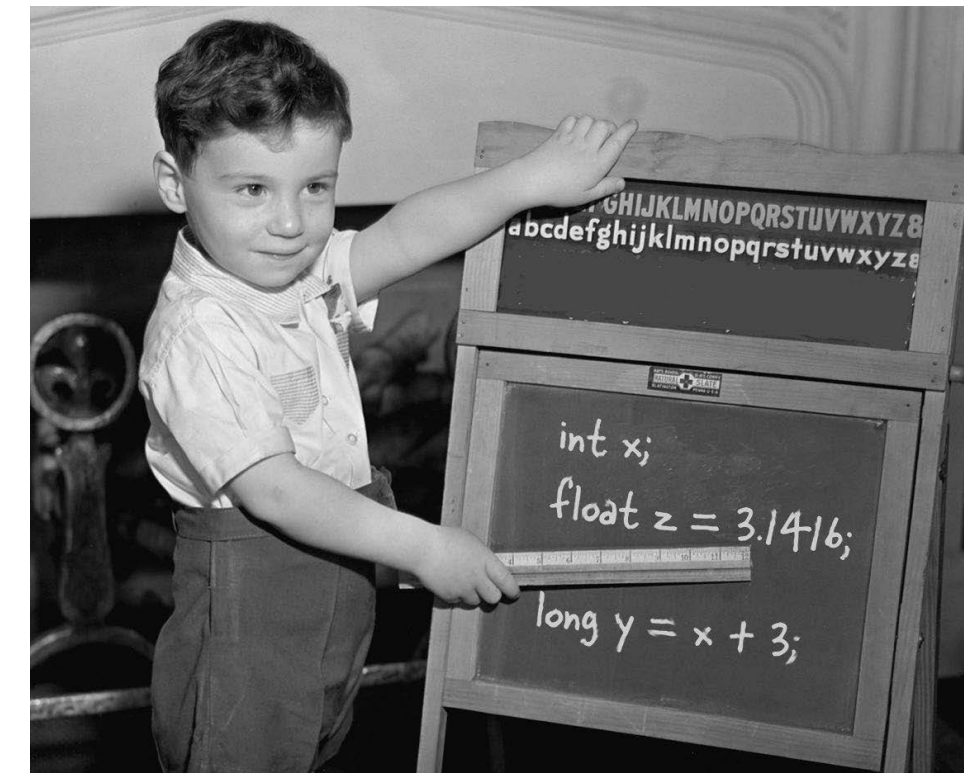
📖 Wo kommen literale Werte in Java vor?

🔑 Welche reservierten Wörter in Java kennen Sie?

👋 Welche Fragen haben Sie zu diesem Kapitel?

3 Elementare Datentypen und Referenzen

Verstehen Sie Ihre Variablen



Variablen gibt es in zwei Geschmacksrichtungen: elementare (oder auch primitive) Typen und Referenztypen. Bis jetzt haben Sie Variablen an zwei Orten verwendet – als Objekt-Zustand (Instanzvariablen) und als **lokale** Variablen (Variablen, die innerhalb einer *Methode* deklariert werden).

Später werden wir Variablen als Argumente (Werte, die *von* aufrufendem Code an eine Methode übergeben werden) und als Rückgabewerte (Werte, die aus einer Methode *an* aufrufenden Code zurückgeliefert werden) nutzen. Wir haben Variablen bisher als einfache, elementare Integer-Werte (Typ `int`) deklariert, Sie haben jedoch auch Variablen gesehen, die als etwas Komplexeres deklariert wurden, zum Beispiel als String oder Array. **Aber das Leben kann doch nicht nur aus Integer-Werten, Strings und Arrays bestehen!** Wie wäre es beispielsweise mit einem `HaustierBesitzer`-Objekt und einer Instanzvariablen vom Typ `Hund`? Oder mit einem `Auto`-Objekt, das einen Motor hat? In diesem Kapitel werden wir die Geheimnisse der Java-Typen lüften (zum Beispiel das Geheimnis des Unterschieds zwischen Elementartypen und Referenzen). Wir werden uns ansehen, was man alles als Variablen deklarieren kann, was Sie in einer Variablen speichern und was Sie mit einer Variablen tun können. Zum Schluss lernen Sie das Leben auf dem Garbage Collectible Heap hautnah kennen.



Typsicherheit

Auf den Typ achten

Typsicherheit

 **Typsicherheit:** Der Datentyp einer Variablen ist festgelegt und kann während der Programmausführung nicht geändert werden.

 Es gibt:

100 **Elementartypen** (oft auch als primitive Datentypen bezeichnet)

 **Objektreferenzen**

 Dies hilft Fehler zu vermeiden und erhöht die Programmsicherheit.

`int counter = 1;`

Java ist der Datentyp wichtig. Sie können keine Giraffen-Referenz in einer Kaninchen-Variablen speichern.



`Giraffe g = new Giraffe();`

Die Programmiersprache Java ist eine **statisch typisierte Sprache**, was bedeutet, dass jede Variable und jeder Ausdruck einen Typ hat, der zur Kompilierungszeit bekannt ist.

Java ist auch eine **stark typisierte Sprache**, weil Typen die Werte begrenzen, die eine Variable enthalten oder ein Ausdruck erzeugen kann, die für diese Werte unterstützten Operationen begrenzen und die Bedeutung der Operationen bestimmen.

Starke statische Typisierung hilft bei der **Erkennung von Fehlern zur Kompilierzeit**.

Statische Typisierung in Java

Typsicherheit

Java ist eine **statisch typisierte** Programmiersprache. Das bedeutet:

- Der Typ jeder Variablen und jedes Ausdrucks wird zur **Kompilierzeit** festgelegt.
- Variablentypen müssen **explizit deklariert** werden oder können vom Compiler durch Typinferenz ermittelt werden.
- Der Compiler kann viele **Typfehler** bereits vor der Ausführung des Programms **erkennen**.

Starke Typisierung in Java

 Typsicherheit

Java ist außerdem eine **stark typisierte** Sprache. Das bedeutet:

- **Strikte Regeln** für Typumwandlungen und Operationen zwischen verschiedenen Datentypen.
- **Beschränkung** der Werte und Operationen, die für einen bestimmten Typ zulässig sind.
- Erhöhte Typsicherheit, da **unbeabsichtigte Typumwandlungen** verhindert werden.

Vorteile der starken statischen Typisierung

Typsicherheit

Die **Kombination** von starker und statischer Typisierung in Java bietet mehrere **Vorteile**:

- **Frühe Fehlererkennung:** Viele Fehler werden bereits zur Kompilierzeit erkannt, was die Fehlersuche beschleunigt.
- **Verbesserte Codequalität:** Der Compiler erzwingt eine strikte Typkonsistenz, was zu robusterem Code führt.
- **Optimierungsmöglichkeiten:** Der Compiler kann den Code aufgrund der bekannten Typen besser optimieren.


```
1 public class TypeSafetyDemo {
2     public static void main(String[] args) {
3         int counter = 1;
4         counter = counter + 1;
5
6         // An integer is not a car, and a car is not an integer, right?
7         counter = new Car();
8
9         Car car = new Car();
10        System.out.print("This car makes: ");
11        car.startEngine();
12
13        // That wouldn't make any sense, would it?
14        counter = counter + car;
15
16        Book book = new Book();
17        System.out.print("This book's title is: ");
18        book.printTitle();
19
20        // A book is not a car, any more than a car is a book, right?
21        book = car;
22
23        // That wouldn't make any sense, would it?
24        book.printTitle();
25    }
26 }
27
```

STDIN

 36 + return

Input for the program (Optional)

Output:

TypeSafetyDemo.java:7: error: incompatible types: Car cannot be converted to int
counter = new Car();
 ^

TypeSafetyDemo.java:14: error: bad operand types for binary operator '+'
counter = counter + car;
 ^

first type: int

second type: Car

TypeSafetyDemo.java:21: error: incompatible types: Car cannot be converted to Book
book = car;
 ^

3 errors

! Quiz

🦒 Typsicherheit

🤔 Ich mache einige Aussagen –
sind diese Aussagen richtig
oder falsch?

👍 Bei **richtigen** Aussagen **stehen**
Sie **kurz auf!**

👎 Bei **falschen** Aussagen **bleiben**
Sie **sitzen!**



Es gibt zwei Arten von Variablen:
elementare Typen und Referenztypen.

Richtig!

In Java ist der Datentyp einer Variablen festgelegt und kann zur Laufzeit nicht geändert werden.

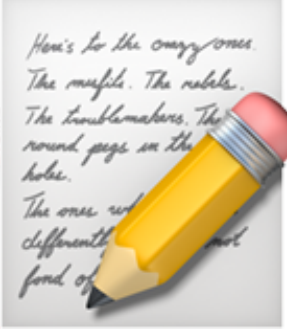
Richtig!

Eine starke Typisierung hat den Vorteil,
dass unbeabsichtigte Typumwandlungen
vermieden werden.

Richtig!

Der Java-Compiler erlaubt weiche
Typkonsistenz, was zu schnellerem Code
führt.

Der Compiler erzwingt eine strikte Typkonsistenz, was zu robusterem Code führt.

 Welcher **eine** Gedanke zu
„ Typsicherheit“ war für Sie
am wichtigsten?  Schreiben
Sie ihn auf!

 Durchatmen und  Sortieren

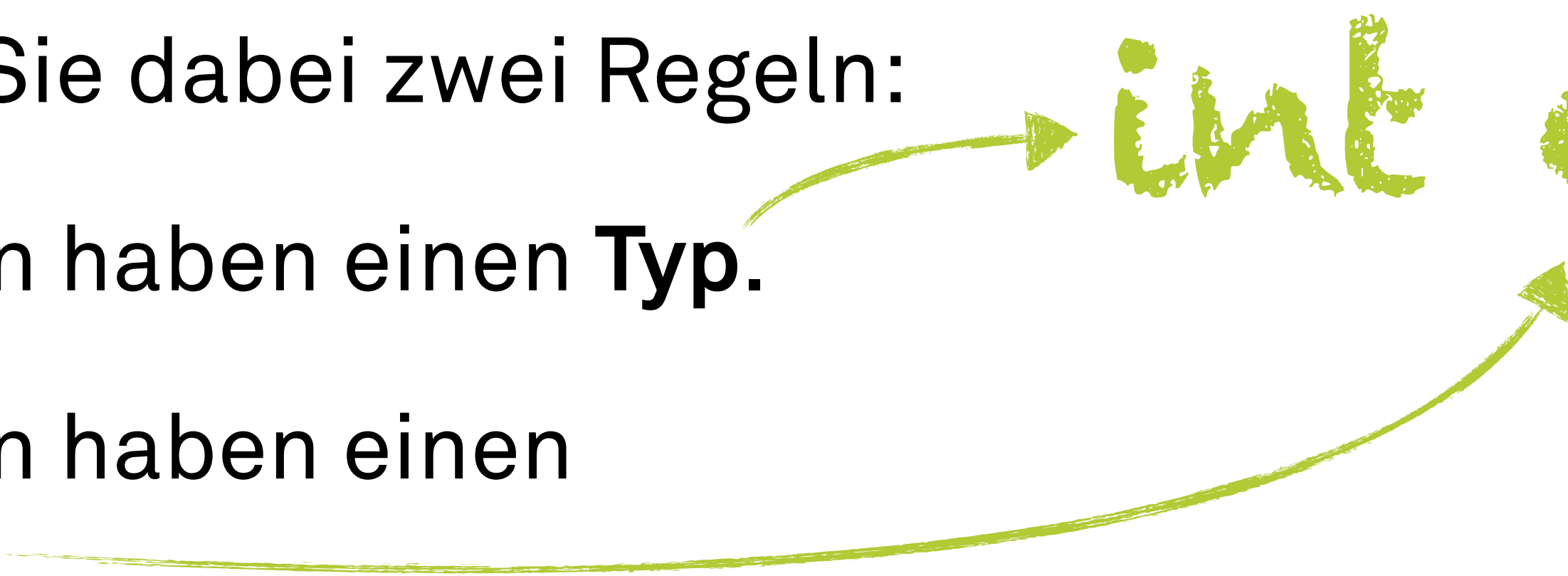
Variablendeklaration

Variablen haben einen Typ und einen Namen

Variablendeklaration

- Damit Java diese Typsicherheit gewährleisten kann, ist es erforderlich, dass **Sie** den **Typ** Ihrer Variablen **deklarieren**.
- Beachten Sie dabei zwei Regeln:
 - Variablen haben einen **Typ**.
 - Variablen haben einen **Namen**.

`int counter;`



! ? Quiz

📧 Variablendeklaration

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



In Java steht es Ihnen frei, den Typ Ihrer Variablen zu deklarieren.

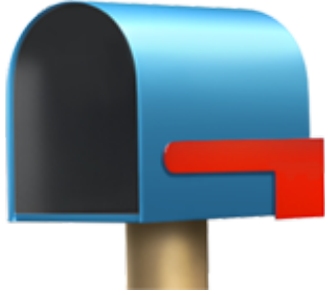
Es ist notwendig, dass Sie den Typ Ihrer Variablen deklarieren.

Variablen müssen immer mit Namen und Typ deklariert werden.

Richtig!

Bei „boolean isActive“ wird boolean als Typ und isActive als Name verwendet.

Richtig!

 Welcher **eine** Gedanke zu
„ Variablendeklaration“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!



Durchatmen und  Sortieren

100 Elementartypen



Variablen als Becher

100 Elementartypen

- Variablen sind wie Becher, wie ein Behälter: sie **enthalten** etwas.
- In diesem Sinne haben sie neben einem Typ auch eine **Größe**:
 - „small“, „short“, „tall“ und „grande“



Ganzzahl-Typen

100 Elementartypen

- In Java haben Elementartypen **verschiedene Größen**.
- Zum Beispiel für die vier ganzzahligen Elementartypen:
 - byte
 - short
 - int
 - long



64 Bit



Wertbereiche

100 Elementartypen

- In Java gibt es eine ganze Reihe von elementaren Typen mit entsprechenden Wertbereichen, zum Beispiel:
- `boolean` `true` oder `false`
- `byte` -128 bis 127
- `short` -32.768 bis 32.767
- `int` -2.147.483.648 bis 2.147.483.647
- `long` „- riesig“ bis „+ riesig“
- Mehr dazu im Buch auf Seite 51.



Nichts verschütten!

100 Elementartypen

- Jeder Elementartyp hat eine bestimmte **Anzahl von Bits**. Das sind sozusagen die passenden **Bechergrößen**.
 - „Ein Lungo passt nicht in eine Espressotasse.“
 - „Ein int passt nicht in ein byte.“
 - int hat „Bechergröße“ 32 Bit.
 - byte hat „Bechergröße“ 8 Bit.
- Mit der Art des Elementartyps wählen Sie auch eine **feste, passende Größe**.
- Mehr dazu auf Seite 52.




```
1 public class LossyConversionDemo {
2     public static void main(String[] args) {
3         // int: -2.147.483.648 to 2.147.483.647
4         // byte: -128 to 127
5
6         int bigNumber = 1_204_266_628;
7
8         // Possible lossy conversion from int to byte is detected.
9         byte smallNumber = bigNumber;
10
11        byte reallySmallNumber = 13;
12        reallySmallNumber = smallNumber;
13    }
14 }
15
```

STDIN

[Return](#)

Input for the program (Optional)

Output:

```
LossyConversionDemo.java:9: error: incompatible types: possible lossy conversion from int to byte
    byte smallNumber = bigNumber;
                        ^
```

1 error

error: compilation failed


```
1 public class OverflowDemo {  
2     public static void main(String[] args) {  
3         // int: -2.147.483.648 to 2.147.483.647  
4         // Max int value minus 1 = 2.147.483.646  
5  
6         int value = 2_147_483_647 - 1;  
7  
8         // What could possibly go wrong?  
9         for (int i = 0; i < 4; i++, value++) {  
10            System.out.println(value);  
11        }  
12    }  
13 }  
14
```

STDIN

Input for the program (Optional)

Output:

```
2147483646  
2147483647  
-2147483648  
-2147483647
```

```
1 public class UnderflowDemo {
2     public static void main(String[] args) {
3         // Math.pow(double a, double b) returns a ^ b as double
4
5         for (int i = 1073; i <= 1076; i++) {
6             double twoRaisedToPower = Math.pow(2, -i);
7             // -> 2 ^ -i
8
9             // What could possibly go wrong?
10            System.out.println("2 ^ -" + i + " = " + twoRaisedToPower);
11        }
12    }
13 }
14
```

STDIN

Input for the program (Optional)

Output:

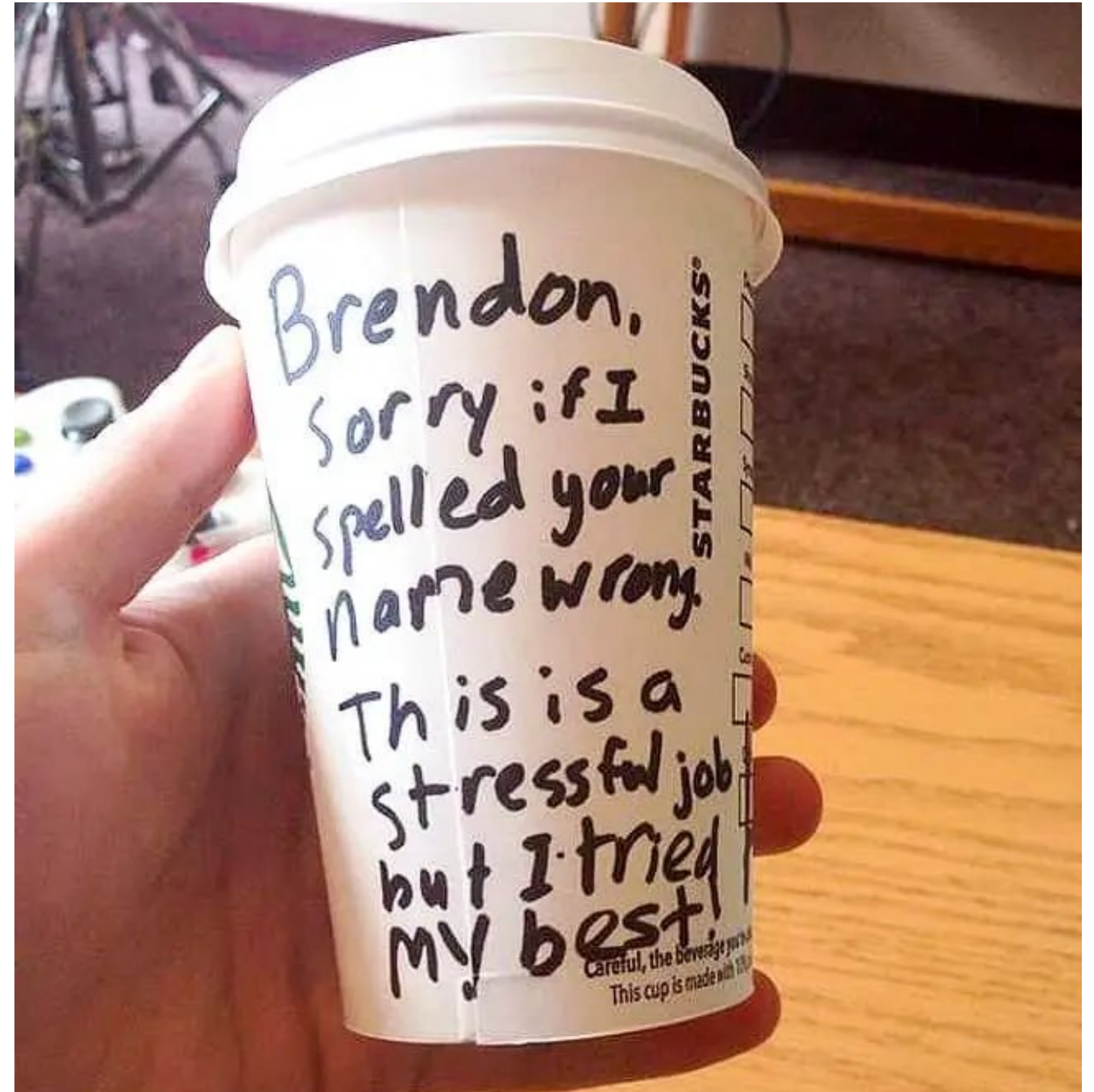
```
2 ^ -1073 = 9.9E-324
2 ^ -1074 = 4.9E-324
2 ^ -1075 = 0.0
2 ^ -1076 = 0.0
```


Becher mit Namen

100 Elementartypen

- Jeder Becher und somit jede Variable enthält einen Wert.
- In Java geben Sie jedem Becher einen **Namen**, ähnlich wie bei Starbucks:
 - „Ich nehme einen `int` mit dem Wert 2486. ~~Ich heiße...~~ Bitte geben Sie dem Becher den Namen `counter`.“

```
int counter = 2486;
```




```
int counter = 2486;
```

Initialisierung ist die Zuweisung eines Wertes an eine Variable zum Zeitpunkt der Deklaration.

🤔 Was ist erlaubt?

💬 Nachbarschaftskonferenz

👥 Tauschen Sie sich mit einer Nachbar:in aus!

? Welche Zuweisungen sind zulässig und welche nicht?

! Lösen Sie die Aufgabe, indem Sie Ihr Wissen über die Größe und den Typ von Variablen mit elementaren Typen anwenden!

🧠 Wir haben nicht alle Regeln behandelt. Ihr logisches Denkvermögen wird Ihnen dabei behilflich sein.

🧙 Der Compiler geht immer auf Nummer sicher.



Was ist erlaubt?

Nachbarschaftskonferenz

 Tauschen Sie sich mit einer Nachbar:in aus!

? Welche Zuweisungen sind zulässig und welche nicht?

! Lösen Sie die Aufgabe, indem Sie Ihr Wissen über die Größe und den Typ von Variablen mit elementaren Typen anwenden!

 Wir haben nicht alle Regeln behandelt. Ihr logisches Denkvermögen wird Ihnen dabei behilflich sein.

 Der Compiler geht immer auf Nummer sicher.

Kreisen Sie in der folgenden Liste die Anweisungen **ein**, die erlaubt wären, wenn diese Zeilen in einer einzelnen Methode stünden:

-  1. `int x = 34.5;`
-  2. `boolean boo = x;`
-  3. `int g = 17;`
-  4. `int y = g;`
-  5. `y = y + 10;`
-  6. `short s;`
-  7. `s = y;`
-  8. `byte b = 3;`
-  9. `byte v = b;`
-  10. `short n = 12;`
-  11. `v = n;`
-  12. `byte k = 128;`



Seite 52


```
1 public class IncompatibleTypesDemo {
2     public static void main(String[] args) {
3         int x = 34.5;    // 1
4         boolean boo = x; // 2
5         int g = 17;      // 3
6         int y = g;       // 4
7         y = y + 10;      // 5
8         short s;        // 6
9         s = y;           // 7
10        byte b = 3;      // 8
11        byte v = b;      // 9
12        short n = 12;    // 10
13        v = n;           // 11
14        byte k = 128;    // 12
15    }
16 }
17
```

STDIN

Input for the program (Optional)

Output:

IncompatibleTypesDemo.java:3: error: incompatible types: possible lossy conversion from double to int
int x = 34.5; // 1
 ^

IncompatibleTypesDemo.java:4: error: incompatible types: int cannot be converted to boolean
boolean boo = x; // 2
 ^

IncompatibleTypesDemo.java:9: error: incompatible types: possible lossy conversion from int to short
s = y; // 7
 ^

IncompatibleTypesDemo.java:13: error: incompatible types: possible lossy conversion from short to byte
v = n; // 11
 ^

IncompatibleTypesDemo.java:14: error: incompatible types: possible lossy conversion from int to byte
byte k = 128; // 12
 ^

5 errors

error: compilation failed

! Quiz

100 Elementartypen

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Variablen sind wie Becher und enthalten in diesem Sinne etwas.

Richtig!

Der Wert einer elementaren Variablen (der „Inhalt“ dieser „Becher“) besteht aus den Bits, die den Wert repräsentieren (5, 'a', true, 3.1416, ...).

Richtig!

In Java sind alle Elementartypen gleich groß.

Sie haben unterschiedliche Größen und damit unterschiedliche Wertbereiche.

In Java hat z. B. `boolean` den Wertbereich `true` oder `false` und `byte` den Wertbereich von -128 bis 127.

Richtig!

 Welcher **eine** Gedanke zu
„100 Elementartypen“ war für
Sie am wichtigsten?
 Schreiben Sie ihn auf!



Durchatmen und



Sortieren



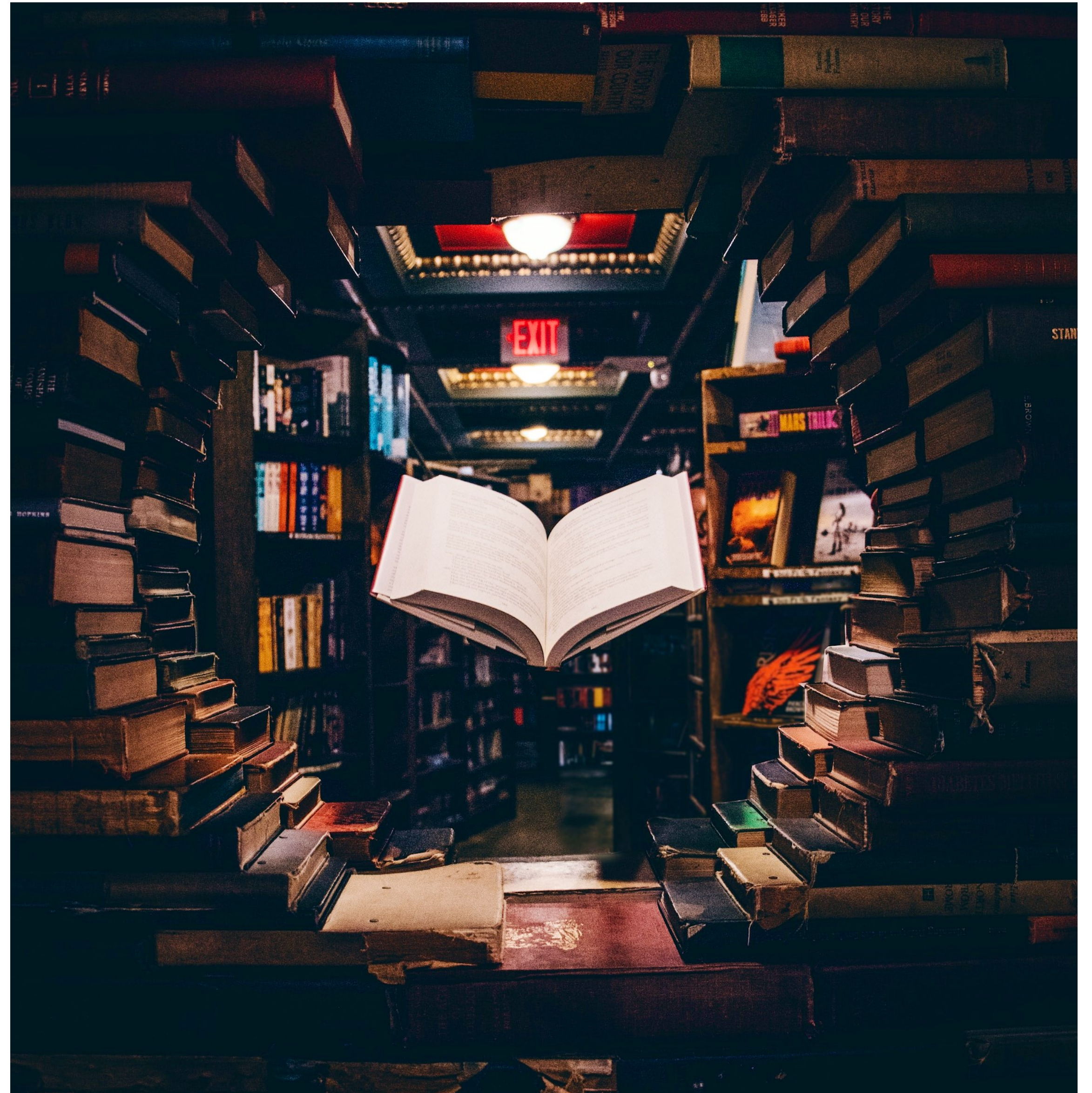
Literale Werte

Buchstäbliche Werte

Literale Werte

- Der Begriff **literal** stammt aus dem Lateinischen und bedeutet wörtlich oder buchstäblich.
- Ein Literal ist die Darstellung eines **festen Wertes** im Quellcode.
 - Literale werden ohne Berechnung **direkt im Code** dargestellt.
- Im Gegensatz dazu können **nicht-literale Werte**, wie Variablen oder Ausdrücke, Werte haben, die erst **zur Laufzeit berechnet** werden.

42



Werte Zuweisen

Literale Werte

- Sie können einer Variablen auf verschiedene Weise einen Wert zuweisen, zum Beispiel:
- Geben Sie nach dem Gleichheitszeichen einen **literalen Wert** an: $x = 42$,
`isLiteral = true`
- Weisen Sie den Wert einer Variablen einer anderen zu: $x = y$
- Verwenden Sie einen Ausdruck, der beides kombiniert: $x = y + 4711$
- Weitere Beispiele auf Seite 52.



! Quiz

📧 Literale Werte

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Ein Literal ist die direkte Darstellung eines festen Wertes im Quellcode, ohne dass eine Berechnung erforderlich ist.

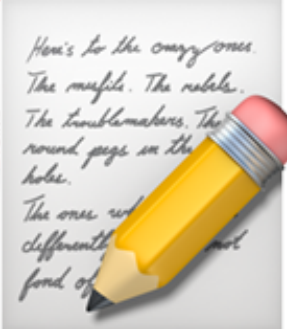
Richtig!

Literale stellen Werte dar, die sich je nach Programmausführung ändern können.

Sie repräsentieren konstante Werte, die sich nicht ändern.

Literale können Variablen direkt zugewiesen oder direkt in Ausdrücken verwendet werden.

Richtig!

 Welcher **eine** Gedanke zu
„ Literale Werte“ war für Sie
am wichtigsten?  Schreiben
Sie ihn auf!

 Durchatmen und  Sortieren

Schlüsselwörter

Regeln für Namen

🔑 Schlüsselwörter

- Sie können eine Klasse, eine Methode oder eine Variable nach folgenden Regeln benennen:
- Der Name muss mit einem Buchstaben, einem Unterstrich (_) oder einem Dollarzeichen (\$) beginnen. Der Name darf nicht mit einer Zahl beginnen.
- Ziffern sind nach dem ersten Zeichen erlaubt. Nur als erstes Zeichen sind sie nicht erlaubt.
- Solange diese beiden Regeln eingehalten werden, kann der Name alles sein. Er darf nur **keines der in Java reservierten Wörter** sein.



Reservierte Wörter

🔑 Schlüsselwörter

- Reservierte Wörter sind **Schlüsselwörter** (und ähnliches).
- Sie sind für die Syntax und Semantik der Programmiersprache reserviert und dürfen daher nicht zur Definition von Variablen (und ähnlichem) verwendet werden.
- Beispiele für Schlüsselwörter in Java sind `class`, `public`, `void`, `int`, `if`, `for` und viele andere.
- Weitere Schlüsselwörter auf Seite 53.



! Quiz

🔑 Schlüsselwörter

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Schlüsselwörter sind in Java reservierte Wörter. Als solche haben sie eine besondere Bedeutung für Syntax und Semantik.

Richtig!

Schlüsselwörter können auch als
Bezeichner (z. B. für Variablen, Methoden
oder Klassen) verwendet werden.

Als Bezeichner sind sie nicht zulässig.

Beispiele für solche Schlüsselwörter sind:
`int, boolean, if, return, public, class`
und `new`.

Richtig!

 Welcher **eine** Gedanke zu
„ Schlüsselwörter“ war für
Sie am wichtigsten?
 Schreiben Sie ihn auf!



Durchatmen und  Sortieren

Pair Programming

🎓 Primitive Typen und Werte

🤝 Pair Programming

- 🎯 Addieren Sie zwei Zufallszahlen und informieren Sie über das Ergebnis.
- ✅ Erzeugen Sie zwei ganzzahlige Zufallszahlen, die jeweils mindestens den Wert 0 und höchstens den Wert 10 haben: $\{0, 1, \dots, 10\}$
- ✅ Addieren Sie diese beiden Zufallszahlen.
- ✅ Ist die Summe kleiner oder gleich 10 (≤ 10), geben Sie eine entsprechende Meldung aus.



Primitive Typen und Werte

Pair Programming

 **Addieren Sie zwei Zufallszahlen und informieren Sie über das Ergebnis.**

-  Erzeugen Sie zwei ganzzahlige Zufallszahlen, die jeweils mindestens den Wert 0 und höchstens den Wert 10 haben: {0, 1, ... 10}
-  Addieren Sie diese beiden Zufallszahlen.
-  Ist die Summe kleiner oder gleich 10 (≤ 10), geben Sie eine entsprechende Meldung aus.

 Ist die Summe größer als 10 (> 10), geben Sie eine entsprechende Meldung aus.

 Nach Ausführung des Programms steht auf der Konsole zum Beispiel entweder Is less than or equal to 10! oder Is greater than 10!.

 Geben Sie Zwischenergebnisse aus!

„Das ist ja spannend, und wie generiere ich
jetzt solche Zufallszahlen? 🤔“

Robin – motivierte und eigenverantwortliche Student:in im 1. Semester



„Ich unterhalte mich mal mit ChatGPT... “

Robin – motivierte und eigenverantwortliche Student:in im 1. Semester





<https://chatgpt.com/share/672f6acc-3658-8001-be1d-3b3aa3309ab3>

„Super, das reicht mir. Den Rest bekomme
ich selbst hin! “

Robin – motivierte und eigenverantwortliche Student:in im 1. Semester



🎓 Primitive Typen und Werte

🤝 Pair Programming

🎯 **Addieren Sie zwei Zufallszahlen und informieren Sie über das Ergebnis.**

✓ Erzeugen Sie zwei ganzzahlige Zufallszahlen, die jeweils mindestens den Wert 0 und höchstens den Wert 10 haben: {0, 1, ... 10}

✓ Addieren Sie diese beiden Zufallszahlen.

✓ Ist die Summe kleiner oder gleich 10 (≤ 10), geben Sie eine entsprechende Meldung aus.

✓ Ist die Summe größer als 10 (> 10), geben Sie eine entsprechende Meldung aus.

🏁 Nach Ausführung des Programms steht auf der Konsole zum Beispiel entweder `Is less than or equal to 10!` oder `Is greater than 10!`.

👤 Geben Sie Zwischenergebnisse aus!

Tipp: Starten Sie mit 2 literalen Werten!

```
import java.util.Random;
```

```
// Erstelle eine Instanz  
// der Random-Klasse
```

```
Random rand = new Random();
```

```
// Generiere eine Zufallszahl  
// zwischen 0  
// (einschließlich) und 11  
// (ausschließlich)
```

```
int zahl = rand.nextInt(11);
```

```
1 public class RandomSumExampleSolutionStep1 {  
2     public static void main(String[] args) {  
3  
4         int randomNumberA = 3;  
5         System.out.println("'Random' number A: " + randomNumberA);  
6         int randomNumberB = 11;  
7         System.out.println("'Random' number B: " + randomNumberB);  
8  
9         int sum = randomNumberA + randomNumberB;  
10        System.out.println("Sum: " + sum);  
11  
12        if (sum <= 10) {  
13            System.out.println("Is less than or equal to 10!");  
14        } else {  
15            System.out.println("Is greater than 10!");  
16        }  
17    }  
18 }  
19
```

STDIN

Input for the program (Optional)

Output:

```
'Random' number A: 3  
'Random' number B: 11  
Sum: 14  
Is greater than 10!
```



```
1 import java.util.Random;
2
3 public class RandomSumExampleSolutionStep2 {
4     public static void main(String[] args) {
5
6         Random generator = new Random();
7
8         int randomNumberA = generator.nextInt(11);
9         System.out.println("Random number A: " + randomNumberA);
10        int randomNumberB = generator.nextInt(11);
11        System.out.println("Random number B: " + randomNumberB);
12
13        int sum = randomNumberA + randomNumberB;
14        System.out.println("Sum: " + sum);
15
16        if (sum <= 10) {
17            System.out.println("Is less than or equal to 10!");
18        } else {
19            System.out.println("Is greater than 10!");
20        }
21    }
22 }
23
```

STDIN

Input for the program (Optional)

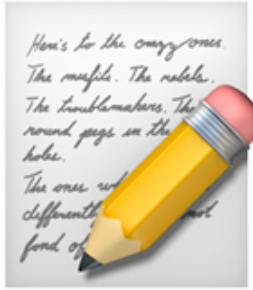
Output:

Random number A: 0

Random number B: 6

Sum: 6

Is less than or equal to 10!

 Welcher **eine** Gedanke zu
„ Primitive Typen und Werte“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren

 **Zielgerade**

↔ Teach Back

📖 Elementare Datentypen und Referenzen

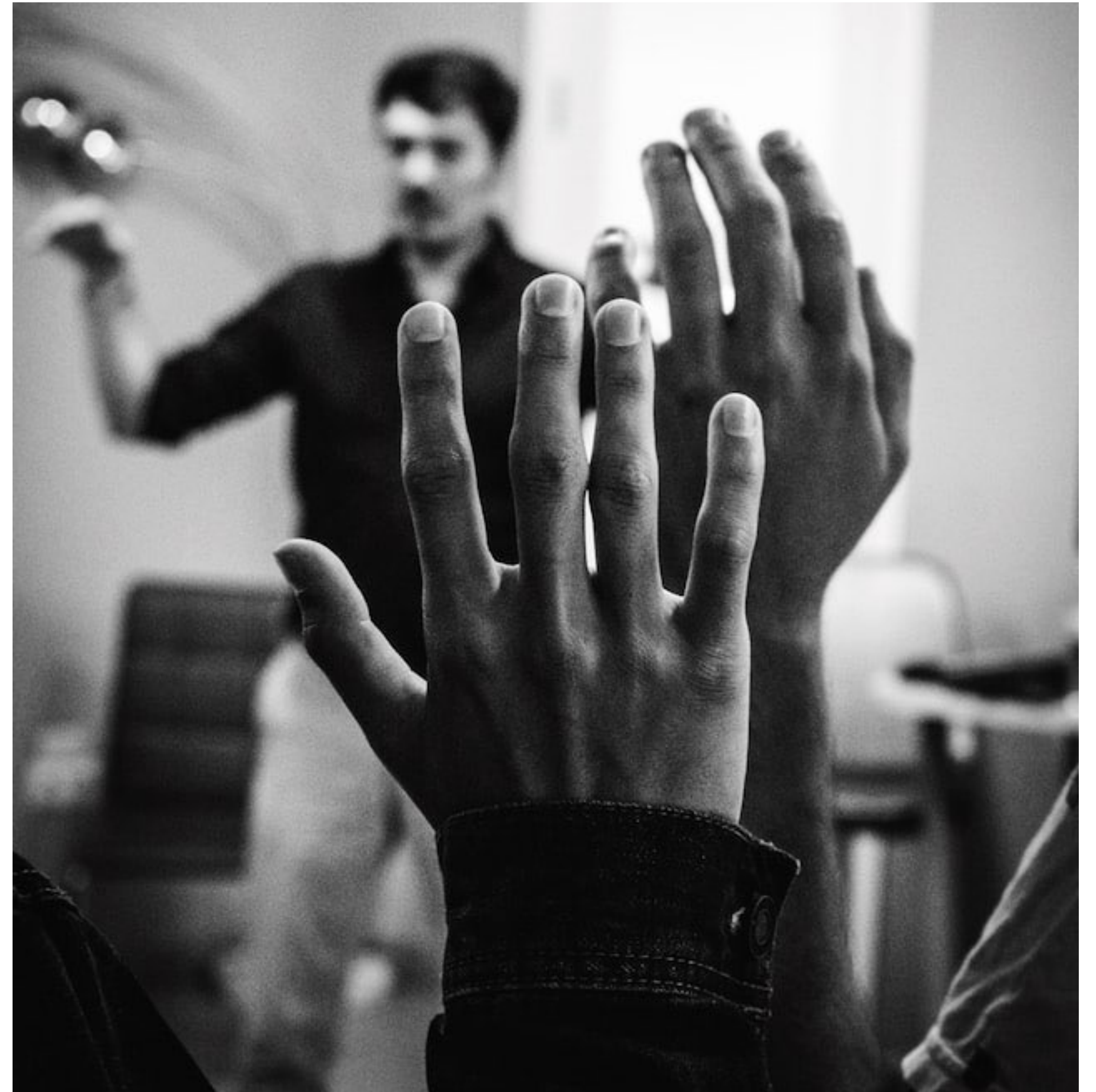
- 🎤 Beantworten Sie sich zusammen mit einer Nachbar:in gegenseitig kurz und knapp die folgenden Fragen:
- 💡 Was habe ich heute gelernt und erfahren?
- 💡 Was davon ist für mich besonders wichtig?



💡 Ihre Fragen

👉 Unsere Antworten

- Welche Fragen haben Sie zu  **Elementare Datentypen und Referenzen?**
- Was irritiert Sie vielleicht sogar?
- Was interessiert Sie außerdem?





Lern-Logbuch



Elementare Datentypen und Referenzen



Machen Sie sich Notizen:



Was sind für mich die wichtigsten Erkenntnisse?



Was weiß ich jetzt, was ich vorher nicht wusste?



Wie kann ich dieses Wissen anwenden?



Welche Fragen habe ich noch?



Was werde ich tun, um sie zu beantworten?



Ihr Lernerfolg – Unser Applaus

